

Lostify AI – Weekly Learning and Execution Plan

1. Project Summary

Lostify AI is an AI-powered image and text similarity search project for lost and found items. The project follows a practical FYP learning flow:

1. Understand AI and computer vision concepts.
2. Use a pretrained CLIP model for image embeddings.
3. Generate vector embeddings for case images.
4. Compare query images with stored cases using cosine similarity.
5. Add metadata and search enhancements.
6. Build an API and retrieval system for real use.

The project is currently at the stage where the model, embeddings, metadata, and fast similarity search have already been explored in Week 01 and Week 02.

2. What the Project Already Contains

Root Structure

- README.md – project overview
- documentation/ – reports, notes, and project documents
- learning/ – all learning modules and implementation work

Learning Folder

- Week-01/
 - README.md – AI fundamentals, CLIP, embeddings, cosine similarity
 - implementation/similarity_test.py – first prototype search test
- Week-02/
 - README.md – precomputed embeddings and metadata workflow
 - generate_embeddings.py – creates .npy embeddings for case images
 - similarity_search.py – loads embeddings and compares query images
 - create_metadata.py – builds metadata.json for matching cases

- build_faiss_index.py – creates FAISS index for fast vector search
- app.py – FastAPI backend for image + text search
- backend/ – backend service and case data
- frontend/ – simple frontend for UI testing
- services/ – AI and related service logic

Current Core Idea

The main architecture is:

Image or text query

↓

CLIP encoder

↓

512-dimensional embedding

↓

Normalize vector

↓

Compare with stored embeddings

↓

Rank most similar lost/found cases

3. What Each Main File Does

README.md

The project overview and entry point for understanding the project direction.

learning/Week-01/README.md

Introduces:

- AI fundamentals
- Computer vision
- Transfer learning
- CLIP models
- Embeddings
- Cosine similarity

learning/Week-02/README.md

Focuses on:

- Precomputed embeddings
- Similarity search pipeline
- Metadata enrichment
- Multimodal search with text + image
- FAISS index creation

generate_embeddings.py

Creates and saves case embeddings as .npy files in the embeddings folder.

similarity_search.py

Loads the saved embeddings and compares them with a query image using cosine similarity.

create_metadata.py

Builds a metadata.json file that gives each case an understandable description and category.

build_faiss_index.py

Builds a FAISS index for quick approximate or exact vector retrieval.

app.py

This is the main backend API. It:

- loads the CLIP model once
- loads the FAISS index
- accepts image and/or text queries
- returns top matching cases with metadata

frontend/

Contains the client-side UI for testing search workflows and showing results.

backend/

Contains the service layer and case data used by the API.

4. What You Should Do First

Priority 1: Understand the Core Pipeline

Read in this order:

1. README.md
2. learning/Week-01/README.md
3. learning/Week-02/README.md
4. app.py
5. similarity_search.py
6. generate_embeddings.py
7. build_faiss_index.py

This gives you the complete mental model before you start editing code.

Priority 2: Run the Existing Flow

Before making changes, run the existing search flow and confirm:

- embeddings are generated
- metadata loads correctly
- search returns results
- the API works locally

Priority 3: Learn the Theory Behind the Code

Focus on these concepts:

- CLIP model
- embeddings
- feature vectors
- normalization
- cosine similarity
- FAISS vector search
- multimodal retrieval

5. Learning Plan for This Week

Day 1 – Understand the project

- Read root README and both Week 01 and Week 02 notes
- Understand how the project is structured
- Write down the main flow in your own words

- Note which files are experimental and which are production-like

Day 2 – Study the model and retrieval logic

- Learn what CLIP is and why it is useful
- Study image embeddings and vector spaces
- Understand cosine similarity and why normalization matters
- Review how `similarity_search.py` works line by line

Day 3 – Study data preparation and metadata

- Inspect `metadata.json` and `create_metadata.py`
- Understand how case data is mapped to visual categories
- Learn why metadata makes results easier to understand

Day 4 – Study the API and FAISS integration

- Read `app.py` and understand request/response flow
- Learn how the index is built and used
- Understand the difference between embedding generation and retrieval

Day 5 – Practice and improve the project

- Run the service locally
- Test an image query and a text query
- Write your own notes on what works and what is missing
- Prepare improvements: better UI, better dataset, improved search logic

Day 6 – Documentation and reflection

- Summarize what you learned in your own words
- Create your own project explanation
- Identify one improvement you want to implement next

Day 7 – Final review

- Review the week's learning
- Check your understanding of the whole system
- Plan the next milestone for Week 03

6. Best Order to Work This Week

1. Read the documentation first
2. Understand the architecture
3. Run the scripts and API
4. Study each file by purpose
5. Practice explaining the pipeline aloud
6. Improve one weak area
7. Write a summary of your findings

This is the best sequence because it prevents jumping into coding before understanding the model architecture.

7. Suggested Weekly Goal

Your goal for this week is not to build everything. Your goal is to understand the base system completely.

By the end of the week, you should be able to explain:

- what the project is doing
- how images are converted into vectors
- how cosine similarity ranks results
- how metadata and FAISS help the search
- how the API receives and returns results

8. Recommended Next Milestone

The next logical milestone after this week is:

- Improve similarity accuracy with better dataset labeling
- Add a better frontend view of search results
- Add case registration and upload flow
- Improve search by combining image and text more intelligently
- Connect the backend to a database for real case records

9. Final Advice

Do not start by trying to build a huge system. Start by learning the current pipeline completely. The project is already strong as a learning project because it shows the full flow:

data -> embeddings -> similarity -> ranking -> API -> result

That is the foundation of your FYP.

If you understand this foundation well, the next features become much easier.